TECHNICAL REPORT

Tradeski: A Real-Time Market Dashboard

Rate-limit budgeting, indicators correct at their boundary conditions, and grounding a language model in one user's actual portfolio.

Daniel Lichtenberger · 2026
tradeski.dev · Source on GitHub

A real-time market dashboard where the interesting engineering is everything except the charts.

Summary

Tradeski streams live prices, macroeconomic series, and scored news into a single browser dashboard, and answers questions about the user's own portfolio rather than the market in general. Plotting a price series is a solved problem. The parts that took real work are the ones a screenshot doesn't show: staying inside third-party API rate limits, computing indicators that are correct at their boundary conditions, and grounding a language model in a specific user's holdings so its answers are about them.

This report describes the architecture as built, the decisions worth defending, and — in §9 — the things it does not do and the places I would build it differently.

Backend	Python 3.12, Flask + Flask-SocketIO on gevent, 31 HTTP endpoints
Frontend	Static HTML/CSS/JS, ~7,000 lines (`dashboard.js` alone is 2,750), Plotly.js, no framework and no build step
Indicators	10, hand-implemented on NumPy arrays — no TA-Lib, no pandas
Tests	304, across 27 files, run in GitHub Actions with flake8
Assistant	Claude Haiku 5.5 with 9 Tradeski tools + web search, a figure-grounding check, and a 24-question eval
Deployment	Backend on Render (gunicorn + gevent), frontend static, Postgres

1. The problem: a dashboard is a rate-limit budget

A market dashboard looks like a data-visualization problem and is actually an API-quota problem. Every panel wants fresh data; every upstream provider meters requests; and the naive implementation — fetch on every page load — exhausts a free-tier news or macro quota within an hour of a handful of users arriving.

So the real design question is not *how do I draw this* but *how stale is each number allowed to be*, and the answer differs per data source by three orders of magnitude. A price can be a minute old. An unemployment rate cannot meaningfully change within a month. Treating those identically is the mistake.

Tradeski assigns every data source its own time-to-live, chosen from how fast the underlying quantity actually moves:

Source	TTL	Why
Price history / stats	5 min	Intraday movement matters; a minute of staleness does not
Screener fundamentals	10 min	Ratios update on filings, not ticks
News + sentiment	30 min	Headline flow is slow; scoring is the expensive part
FRED macro series	1 hour	Most series are monthly or quarterly
Correlation matrix	1 hour	Computed over 90 days; a fresh day barely moves it

The cache itself is deliberately unremarkable — a dict of `(value, expires_at)` with lazy eviction on read (`cache.py`). Its limitations are real and are discussed in §9. What matters is that the TTL is a per-source decision rather than a global constant, and that per-route rate limiting sits in front of the expensive endpoints: registration at 10/hour, login at 10/minute, the assistant at 20/hour and 50/day.

The screener used to fan out across a 29-symbol universe through an eight-worker thread pool. That was a demo, not a screener, and it could not grow: every symbol was another request from a 512 MB free-tier server that Yahoo often refuses. It now covers every US-listed common stock (about 5,500) by moving the work out of the request path entirely. A GitHub Actions job runs after each close. It reads Nasdaq Trader's public symbol directory, downloads a year of daily prices for the whole universe in batches of 200, computes every technical column itself, and refreshes fundamentals for the 1,000 companies whose figures are oldest, large caps first. The first full run showed why that cap exists: Yahoo answered about 1,000 company lookups from one address and then refused, so the job backs off once, then stops and keeps the previous values. Fundamentals change quarterly, so a six-day turnover costs nothing. Ratios that move with the price (P/E, market cap, P/S, P/B, yield, target upside) are recomputed from that day's close. The output is published as one static file on a data branch whose single commit is replaced each run, so a few megabytes of daily prices never accumulate in git history. The browser downloads it once and filters locally: about 40 filters over 5,500 rows takes milliseconds, and the backend is never involved.

Saved screens turn that into an alert without a scheduler. Each saved screen stores the symbols it matched when it was last opened (on the account through `PUT /screens`, or in the browser when signed out). On the next visit after a rebuild, the browser re-runs every saved screen over the new file and counts symbols outside that list: the Screener button shows the total, and opening the screen tags those rows and makes today's matches the new baseline. Screens matching more than 3,000 stocks are not tracked, since "new" means little there and the list would dominate the stored document.

---

2. Architecture

```
browser (static HTML/JS, Plotly.js)
    |
    ├── HTTP  ──►  Flask app  ──►  SimpleCache  ──►  upstream APIs
    |                  |                             (prices, FRED, NewsAPI)
    |                  ├── Postgres  (users, portfolios, alerts, price history)
    |                  └── Anthropic API  (assistant, grounded — §7)
    |
    └── Socket.IO  ◄──  background tracker task (price_update events)
```

The frontend holds all UI state and is served as static files; the backend is a pure JSON and event layer with no templating. That split is what allows the frontend to deploy independently of the backend, and it keeps the server free of rendering concerns entirely.

Two entry-point details are load-bearing. `wsgi.py` calls `gevent.monkey.patch_all()` as its literal first statement, before any other import — patching the standard library's sockets after `requests` has already bound them produces a server that appears to work and then deadlocks under concurrency. And because the backend package directory contains a space in its name (`Plotly dashboard/`), it cannot be imported as a module; `wsgi.py` inserts it onto `sys.path` explicitly so the process works regardless of working directory.

---

3. Indicators from first principles

Every technical indicator is implemented directly on NumPy arrays rather than pulled from TA-Lib or pandas: SMA, EMA, RSI with Wilder smoothing, MACD, Bollinger Bands, ATR, the Stochastic Oscillator, rolling Z-score and rolling volatility (`analyzer.py`). A least-squares "next value" extrapolation is still in the module, but it is no longer computed for the dashboard (see §9).

The point of writing them by hand is not that the libraries are wrong. It is that the boundary conditions are where indicators are actually wrong, and you only meet them if you write the code.

RSI when there are no losses. Relative Strength is `avg_gain / avg_loss` . When the average loss is zero, RS diverges and the standard formula divides by zero. The common shortcut is to set `rs = 0` , which inverts the meaning: a window of pure gains — maximally overbought — reports RSI 0, maximally *oversold*. Tradeski handles the case explicitly: RSI is 100 when there were gains and no losses, and a neutral 50 for a perfectly flat window where both averages are zero.

The Stochastic Oscillator needs highs and lows. %K is defined against the intraday high and low of the lookback window. The real-time tracker stores closing prices only. Feeding the close in as all three of high, low and close makes the formula degenerate — the numerator and denominator collapse to the

same quantity and %K pins to a constant. Rather than emit a plausible-looking number, `analyze_series` returns `stoch_k` and `stoch_d` as `None` when highs and lows were not supplied, and the dashboard omits the panel. A missing indicator is honest; a fabricated one is not.

---

4. Portfolio risk metrics

Risk metrics are computed on the user's actual holdings, weighted by current market value, from a year of daily returns.

The portfolio return series is built as the value-weighted sum of each holding's daily returns, aligned to a common length so that a symbol with a shorter history does not silently shift the series. From that: annualized volatility as the daily standard deviation scaled by `√252`; the Sharpe ratio against a 4.5% annual risk-free rate converted to a daily figure; and beta as `cov(portfolio, SPY) / var(SPY)`.

Two guards matter. A portfolio with zero return variance — a single holding entered today, or an untraded position — makes the Sharpe denominator zero, so the endpoint reports that it cannot be computed rather than dividing. And beta falls back to 1.0 if SPY's variance is degenerate. Both return an explicit reason to the client instead of a number.

Correlation is computed on aligned daily *returns*, not price levels, which would report near-1.0 correlation for any two assets that both drift upward. It is available through Ski's correlation tool for any tickers, the user's holdings included. An earlier heatmap showed the same calculation for a fixed list of 11 symbols, whatever the user owned or was looking at. It answered a question nobody had asked, so it was removed.

Options: priced here, not taken from the feed

The Options page (`/options`, maths in `tracker/options.py`) shows a chain, builds strategies and scans for cash-secured puts and covered calls. Its numbers follow the same rule as Ski's tools: the code computes them, and nothing is taken on trust from the feed.

The data source does report an implied volatility per contract, but out of hours it is often stale, and for deep in-the-money strikes it is close to meaningless. So Tradeski solves IV itself, by bisection on Black-Scholes, from each contract's bid/ask midpoint. That uses the dividend yield and the 3-month Treasury yield from FRED, and time runs to 4pm New York on the expiry date. A midpoint outside the no-arbitrage bounds (below intrinsic value, or above the stock price) gets no IV rather than a nonsense one. The feed's figure is used only as a labelled fallback when there is no two-sided quote. The volatility curve plots only the out-of-the-money side at each strike, because in-the-money quotes are mostly intrinsic value.

A strategy's payoff at expiry is piecewise linear, so its extremes and break-evens are exact. Max profit and loss are read at zero and at each strike. The slope above the top strike says whether either is unlimited. Break-evens are the roots on each segment. "Chance of profit" integrates a lognormal at the ATM implied volatility over the price intervals where the payoff is positive. That is the probability the market's prices imply under risk-neutral drift, and the window says so: it is not a forecast. The same figure appears in the scanner and the builder for the same trade. An earlier version used the contract's own IV in one and the ATM IV in the other, and showed 73% and 76% for the same put.

The page is built to answer "which strategy, and what happens if…?" rather than only "what does this strategy do?". Given a target price, `compare_strategies` prices all nine presets and ranks them. Positions that make money at the target come first, ordered by profit at the target per dollar they can lose. Positions with unlimited loss come last among equals, because a high payoff at one price says nothing about the tail. An `iv_shift` re-values the position before expiry with every option's IV moved by that many points. This isolates the effect beginners most often miss: around earnings, IV routinely falls 10 to 30 points. In the AAPL example, a straddle held over earnings that cost $4,078 was worth $2,133 less after a 15-point drop, with the stock unchanged. The next earnings date comes from Yahoo's chain response, using its estimate of the next report rather than its timestamp of the last one.

A paper trade is re-analysed *as held*: its legs keep their entry prices while volatility comes from today's chain. Its break-evens and chance of profit are therefore measured from what was actually paid, not from what a new position would cost today. Those two answers differ as soon as the stock moves. Ski can call the same ranking (`rank_option_strategies`) when someone states a view, and is told to explain the trade-off, never to recommend one.

---

5. News sentiment: why general-purpose scoring fails here

Sentiment is VADER, extended with roughly 45 finance-specific terms (`news.py`).

The extension is not decoration. VADER is tuned on social-media English, and financial headlines use a register it has no entries for. "Apple beats expectations" scores 0.0 out of the box — *beats* is not in the lexicon in its financial sense, and the headline reads as neutral. Likewise "misses", "downgrade", "buyback", "record high". The single most important event type in equity news is invisible to the default model.

So the lexicon adds the vocabulary with hand-assigned polarities: `beats +2.0`, `all-time high +2.2`, `downgrade -1.8`, `bankruptcy -3.0`, `tariffs -1.2`. Headlines are scored on title plus description, aggregated to a mean compound score with bullish / neutral / bearish counts at a ±0.05 threshold.

Query construction matters too. NewsAPI searched for the bare ticker `META` returns articles about metadata and metaphysics. The first fix, a descriptive query like `"Apple stock"`, was not enough: NewsAPI's default search covers the whole article body, and sorted by date it returned whatever mentioned Apple anywhere. One live check of AAPL turned up a political column, a list of iOS app deals

and a history piece. The query is now the company's name (from the dashboard's ticker index, so any listed company has one) or its ticker, matched in the **title**, first on about twenty financial news sites. Only when those return fewer than four headlines does the search widen to any source, still by title. One-letter and two-letter tickers are left out of the query, because "ALL", "IT" and "ON" are words.

This is a hand-tuned heuristic, not a validated model. The polarities are my judgment, not fitted values, and there is no labeled financial-headline set here measuring whether the extended lexicon actually beats the default. It plainly fixes the specific failures above; the general claim is untested, and §9 says so.

That is also why the assistant (§7) no longer leads with the score. It is shown each headline with its source and date, and the VADER aggregate comes last, labelled as a rough hint: a language model reading "Tesla deliveries fall short of estimates" judges it better than a word list does, and putting the score first anchored its answers to the score.

---

6. Security

Passwords are hashed with PBKDF2 via werkzeug. Sessions are signed bearer tokens issued with `itsdangerous` under a versioned salt, so tokens can be invalidated as a class by rotating it. Every portfolio, alert, and watchlist query is scoped by `user_id` at the database layer rather than filtered after retrieval — the difference between a bug that returns nothing and a bug that returns another user's holdings. CORS is restricted to an explicit origin allowlist from the environment rather than a wildcard.

---

7. The assistant, and what "grounded" means

The dashboard includes Ski, a Q&A assistant backed by Claude Haiku 5.5. The engineering that makes it useful is not the prompt. It is what the model is given, what it can look up, and whether anyone checks what it says.

A general financial chatbot can explain what a P/E ratio is. That is worth very little inside a dashboard, because the user is looking at their own screen and their question is about *their* position. So each request starts with a context block: the user's holdings with weights, P&L and risk metrics, the live FRED snapshot, recent headlines for the charted symbol, and the chart's latest indicator values.

The first version stopped there, and that made it a wrapper. Everything Ski knew was one paste of text. Asked about NVDA while viewing AAPL, it had no NVDA data at all. It saw "RSI 64" with no way to tell whether that was rising or falling. Worse, the prompt claimed data it never received (ATR, stochastic, alerts) and taught it to read VIX and options flow that were never supplied, which invites invented numbers. It also called MACD a "bullish crossover" whenever MACD sat above its signal line, which is wrong: a crossover is the bar where the histogram changes sign.

Now the model decides what to look up, and code computes every number (`ski_tools.py`). Ski has six tools built on the same cached fetchers and hand-written indicators as the dashboard:

- a price summary for any ticker (returns, distance from the 52-week range, realised volatility, and indicators now and five days ago), with its excess return over the S&P 500 and its beta, so "is it the stock or the market?" is answered by code, not by the model subtracting two numbers
- fundamentals and headlines for any ticker
- a what-if that recomputes weights, Sharpe, beta and volatility after hypothetical trades, and reports the change in each
- return correlation aligned on common trading days
- the recent history of a FRED series

Fundamentals were blank in production, and the fix computes them from filings. Yahoo's fundamentals endpoint is blocked from Render, so all 29 screener rows had no P/E or market cap, and Ski's fundamentals tool returned empty fields (and a "—" placeholder as the sector). Blank fields are now dropped, and Ski is told to say so when P/E is unavailable. With `SEC_USER_AGENT` set, `sec_fundamentals.py` computes trailing EPS, P/E, market cap, revenue growth and net margin from SEC EDGAR filings. 10-K filings report only the full year, so the fourth quarter is derived as the year minus its first three quarters, and a window with a missing quarter is refused rather than summed. The screener table uses the same figures for its blank P/E and market cap columns, but only from cache: one throttled background warm-up fetches the filings, so the page never waits on EDGAR. Each reply also lists what Ski looked up ("NVDA price and chart · web search"), so a reader can tell data from prose.

Web search covers what the tools don't. The loop allows at most four rounds of tool calls, then requires an answer. If the API rejects a tool feature, the loop drops web search, then all tools, rather than failing the chat.

Grounding is checked, not assumed (`ski_grounding.py`). After each answer, every figure in the reply is matched, at the reply's own rounding, against the numbers Ski was actually shown. A figure that matches nothing is either invented or calculated by the model itself, and is logged. This is a heuristic. It skips counts, dates and indicator lengths, and it needed a rule for textbook levels: the first live answer was flagged for "70" in "approaching overbought (typically 70+)", which is a rule of thumb, not data. For now it only logs.

Quality is meant to be measured, not asserted. `tests/ski_eval/` asks 23 questions through the real endpoint against fixed fixture data (including a fixture option chain), in three groups: lookups, portfolio risk, and honesty traps (short interest, trade-level options flow, tomorrow's close). The put/call ratio used to be a trap; with the options tools it is now a lookup. Grading is deterministic, with no model judging. As of this writing the eval has not been run against the live model, so there is no score to report; §9 says so.

---

8. Testing

304 tests across 27 files run on every push through GitHub Actions, alongside flake8 linting. Coverage is weighted toward the code where correctness is checkable rather than the code that is merely tedious: indicator boundary conditions, authentication and token handling, database access and user scoping, the FRED and news clients against recorded payloads, the screener, and portfolio math.

Network-dependent modules are tested against fixtures, not live APIs. A test suite that fails because a third-party service is down teaches you nothing and trains you to ignore it.

That style of testing has a blind spot, and it cost this project its headline feature. The background tracker emitted `price_update` on the `/stream` namespace while the browser connected to the default namespace and listened there. Each half was correct read on its own; the socket connected, the UI reported a healthy connection, and no tick ever arrived — prices moved only on the 60-second polling refresh. Every test passed the entire time, because every test exercised a pure function in isolation. A contract *between* two components is invisible to that.

`tests/test_stream_contract.py` now parses `app.py` and `dashboard.js` and asserts that every event the client subscribes to is emitted on the namespace the client is actually on. Worth recording that the first version of that test also passed with the bug present: it compared the *set* of namespaces each event was emitted on, and a dead code path that emitted `price_update` correctly on the default namespace masked the live one that did not. Checking each emit site individually, rather than unioning them per event, is the whole difference between a test that catches this and a test that looks like it does.

The same blind spot hid a second dead feature, found much later. **Alerts never fired.** The form saved them, the panel listed them, and nothing ever checked them: the function that would broadcast a triggered alert was defined and never called, and the background tracker only pushed prices. The stream contract test passed throughout, because that dead function was the "somewhere" it found `alert_triggered` emitted. A test that asks whether code exists cannot tell you whether it runs. Alerts are now checked by `alert_eval.py` against the price history since each alert was set, which also suits a backend that sleeps: a crossing that happened while it was asleep is caught on the next check, with the time it happened. `tests/test_alerts.py` asserts that an alert actually fires, end to end through `/alerts`, which is the property the old tests never checked.

---

9. Limitations

Stated plainly, because a report that lists only strengths is marketing.

The cache is per-process and in-memory. It does not survive a restart, and under multiple gunicorn workers each holds an independent copy — so the effective upstream request rate is the intended rate multiplied by the worker count, and two users can see figures that differ by one TTL. Correct for a single-

worker deployment, which is what runs; wrong the moment it scales horizontally. Redis is the obvious fix and is not implemented.

The risk-free rate is the fed funds rate, which is an approximation. The Sharpe ratio used to subtract a hardcoded 4.5%, so it was wrong by however far rates had moved. It now uses the effective federal funds rate from the FRED snapshot the dashboard already fetches, falls back to 4.5% only when FRED is unavailable, and reports which rate it used. A three-month Treasury bill yield would be the textbook choice.

In production, option quotes are end-of-day. Yahoo refuses option requests from Render's servers (as it does company fundamentals) but answers GitHub's runners, so the daily snapshot job saves closing chains for twelve symbols and the backend falls back to them. The window says so and dates them. Any other symbol has no options on the live site until a data source that serves this host is wired in. A trimmed copy of each day's chains is kept in `data/options-archive/`, a history of real quotes that no free source offers; AlphaGlyph's options backtester checks its model prices against it.

Options values are model values. Black-Scholes treats every option as European. US equity options are American, so values are approximate where early exercise matters: deep in-the-money puts, and calls just before an ex-dividend date. Quotes are delayed about 15 minutes and priced at the midpoint, and real fills are usually worse. The builder reports what crossing the spread would cost, but no slippage beyond that. No free source has IV history, so Tradeski records its own after each close, starting 2026-11-06. For the first 20 trading days there is no percentile at all, and for the first year it covers less than a year, so it is labelled with its length rather than called an IV rank.

Sentiment is unvalidated, and is now labelled that way. See §5. The lexicon extension is a fix for named failures, not a measured improvement, so the news panel shows "Headline tone" (positive, neutral, negative) with a note that it is a word-list score, not a call on the stock.

The assistant's quality is unmeasured. The eval in §7 exists and its harness is tested end to end with a scripted model, but it has not yet been run against the real one. Until it is, "Ski answers better with tools" is a design argument, not a result. The grounding check is a heuristic too: an invented figure that happens to equal a number in the data, or a textbook level, gets through it.

There is no backtesting and no strategy evaluation. Tradeski shows you the present. The question of whether any indicator on the screen predicts anything is deliberately out of scope here — that question is the entire subject of AlphaGlyph, and the honest answer it arrives at is mostly "no". Reading the two projects together is the intended experience: this one displays the indicators, that one measures whether displaying them means anything.

Dead code has been removed, and it hid a dead feature. `tracker/main.py`, a CLI tracker from before the web dashboard that no longer imported, is gone, with the four modules only it used (the old tick-based alert rules, a scheduler, a daily summary and a pruning job) and the `apscheduler` dependency. The old

alert rules looked like the dashboard's alert engine, which made it easy to assume alerts worked; they did not (§8).

The linear-regression "prediction" is no longer sent. It fitted a least-squares line and read off the next point: a trend extrapolation with no error bars and no test against the naive "tomorrow equals today" baseline. The dashboard had stopped displaying it, yet `/price_history` still computed and shipped it. It is no longer computed there.

---

10. What I would do differently

Move the cache to Redis and make TTLs configuration rather than constants. Validate the sentiment lexicon against a labelled set of headlines, so "tone" could become a measured claim. Test features end to end from the first day: alerts were broken for months, and every unit test passed the whole time.

For the assistant, I would have written the eval before the first prompt rather than after the feature shipped. Until then, Ski was judged by reading answers and deciding they looked right, and that is how a prompt claiming data it never received went unnoticed.